计算机网络第二次实验：设计可靠传输协议并编程实现

邹佳衡 2312322 密码与网络空间安全学院

- 计算机网络第二次实验：设计可靠传输协议并编程实现
  - 1. 协议设计说明
  - 2. 实现方法概述
  - 3. 实验步骤与测量方法
  - 4. 结果分析
    - 4.1 发送窗口大小影响（服务端窗口固定32）
    - 4.2 不同接收窗口测试（客户端窗口固定32）
    - 4.3 传输文件实战
  - 5. 具体实现详解
    - 5.1 连接管理
      - 5.1.1 三次握手建立连接
      - 5.1.2 四次挥手断开连接
      - 5.1.3 连接异常处理
    - 5.2 校验和
    - 5.3 确认重传
      - 5.3.1 流水线
      - 5.3.2 选择确认
      - 5.3.3 重传策略
    - 5.4 流量控制
    - 5.5 RENO算法

1. 协议设计说明

- 目标：在用户空间基于 UDP 实现面向连接的可靠数据传输（单向数据流，控制信息双向交互）。
- 报文头（RDTHeader，按 1 字节对齐）：总共20个字节，是一个紧凑的、面向连接的、支持 SACK 和滑动窗口的可靠传输协议头。它包含了序列控制、确认机制、流量控制和错误检测所需的所有字段，适用于在 UDP 上实现可靠传输。
  - uint32 seq：序列号（基于字节），指向数据段起始字节偏移。
  - uint32 ack：确认号，为接收端期望的下一个字节（即已收到的下一个字节位置）。
  - uint16 flags：标志位，含 RDT_SYN/RDT_ACK/RDT_FIN。
  - uint16 checksum：16-bit 校验和（对网络序头部 + 负载计算）。
  - uint16 wnd：接收窗口（以段为单位或字节，当前实现为段数）。
  - uint16 len：负载长度（字节）。
  - uint32 sack：选择确认位图（针对接收窗口之后的 32 段）。

```
 0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7
+---------------+---------------+---------------+--------------
|                         seq:序列号                            |
|                                                               |
+---------------+---------------+---------------+--------------
```

```
|                        ack:确认号                                |
|                                                                 |
+---------------+---------------+---------------+---------------
|            flags              |            checksum             |
|            标志位             |           16-bit 校验和         |
+---------------+---------------+---------------+---------------
|              wnd              |              len                |
|            接收窗口           |             负载长度            |
+---------------+---------------+---------------+---------------
|                        sack:选择确认位图                        |
|                                                                 |
+---------------+---------------+---------------+---------------
```

```
#define RDT_SYN 0x01
#define RDT_ACK 0x02
#define RDT_FIN 0x04
#define RDT_RST 0x08
#define RDT_MSS 1400
#define RDT_MAX_SACK_BITS 32

#pragma pack(push, 1)
struct RDTHeader {
    uint32_t seq;      // sequence number (byte-based)
    uint32_t ack;      // ack number (next expected)
    uint16_t flags;    // SYN/ACK/FIN
    uint16_t checksum; // simple 16-bit checksum
    uint16_t wnd;      // receiver window (in segments)
    uint16_t len;      // data length
    uint32_t sack;     // selective-ack bitmap for next RDT_MAX_SACK_BITS packets
};
```

- 连接管理：3 次握手建立（SYN -> SYN+ACK -> ACK），关闭由 FIN 发起并由对端返回 ACK。
- 差错检测：发送方填充 checksum，接收方在原始网络字节上调用 verify_checksum_raw 校验。
- 确认与重传：基于字节序号的流水线传输，接收端发送累积确认（ack）与 SACK 位图用于选择确认。
- 流量控制：发送方受接收端 wnd 与自身 cwnd（拥塞窗口）共同限制。实验中 wnd 为固定窗口。
- 拥塞控制（RENO）：慢启动（cwnd 以指数增长）、拥塞避免（线性增长）、3 个重复 ACK 触发快速重传并将 ssthresh = cwnd/2；超时触发慢启动（cwnd=1）。

2. 实现方法概述

- 代码结构（高层）：
  - RDT.h：公用数据结构、字节序与校验和函数、辅助函数。
  - client.cpp：发送端实现，包含分片、发送循环、重传逻辑、拥塞控制、接收线程处理 ACK。
  - server.cpp：接收端实现，包含握手、接收/缓存乱序段、发送 ACK+SACK、写入文件。
- 线程模型：发送端使用单独的接收线程处理 UDP ACK（调用 recvfrom、解析并更新发送窗口）；主线程负责分片、发送与定时重传。

- 数据结构：每个分段用 Packet 结构保存 RDTHeader、payload、发送时间与是否已确认标志。
- 校验和：发送前将 header 转换为网络序（hdr_hton），计算 checksum（fill_checksum）；接收时在原始网络字节上调用 verify_checksum_raw。
- 超时与重传：维护每包 sent_time，若超时（RTO）则重置 ssthresh 并将 cwnd=1，并重发最早未确认分段。
- SACK：接收端为下一个期望 ack 之后的 32 段构造位图，发送给发送端用于选择性确认。
- 运行环境与接口：使用 Winsock2 API (socket/sendto/recvfrom/bind)；客户端接受命令行参数 --mss、--wnd、--rto 与 --loss。

3. 实验步骤与测量方法

- 编译并确认可执行文件 RDTServer.exe 与 RDTClient.exe 可在同一机器上运行。
- 使用`fustil file createnew input.bin 31457280` 生成一个 30MB的二进制文件，用作测试
- 运行命令测试三种不同的情况下的吞吐率和传输时间：相同接收端口不同发送端口、相同发送端口不同接收端口、不同丢包率。每组测三次取平均值。

4. 结果分析

4.1 发送窗口大小影响（服务端窗口固定32）

接收窗口大小	测试序号	传输时间(ms)	吞吐率(MB/s)	平均吞吐率(MB/s)	排名
1	测试1	18,857	1.6682	1.68074	4
	测试2	17,855	1.76182		
	测试3	19,512	1.6122		
4	测试1	17,569	1.7905	1.70697	2
	测试2	19,068	1.64974		
	测试3	18,717	1.68068		
8	测试1	18,215	1.7270	1.70249	3
	测试2	18,843	1.66944		
	测试3	18,385	1.71103		
16	测试1	19,787	1.5898	1.62749	5
	测试2	19,105	1.64655		
	测试3	19,110	1.64612		
32	测试1	18,980	1.65739	1.74828	1
	测试2	17,710	1.77624		
	测试3	17,368	1.81122		

接收窗口大小	测试序号	传输时间(ms)	吞吐率(MB/s)	平均吞吐率(MB/s)	排名
64	测试1	20,004	1.57255	1.60557	6
	测试2	18,633	1.68826		
	测试3	20,218	1.5559		
128	测试1	22,679	1.38707	1.42376	7
	测试2	22,096	1.42366		
	测试3	21,538	1.46055		

1.最佳性能窗口

最佳点：发送窗口=32（平均吞吐量 1.74828 MB/s）

这正好匹配服务端的接收窗口大小（32段）

说明当发送窗口等于接收窗口时，管道能达到最佳填充

2.窗口大小的影响规律

窗口太小（1-8）： 吞吐量受限于发送窗口，无法充分利用网络容量

窗口适中（32）： 最佳匹配，充分利用接收方能力

窗口过大（64-128）： 吞吐量反而下降，原因可能是：

- 过多的在途数据可能导致拥塞
- 乱序到达增加服务器缓冲压力
- 可能触发不必要的重传

4.2 不同接收窗口测试（客户端窗口固定32）

接收窗口大小	测试序号	传输时间(ms)	吞吐率(MB/s)	平均吞吐率(MB/s)	排名
1	测试1	77,330	0.406793	0.405476	7
	测试2	76,723	0.410011		
	测试3	78,717	0.399625		
4	测试1	26,525	1.18595	1.18923	6
	测试2	26,556	1.18456		
	测试3	26,276	1.19719		
8	测试1	19,434	1.61867	1.65527	2
	测试2	18,129	1.73519		
	测试3	19,515	1.61195		

接收窗口大小	测试序号	传输时间(ms)	吞吐率(MB/s)	平均吞吐率(MB/s)	排名
16	测试1	18,078	1.74009	1.78216	1
	测试2	17,985	1.74908		
	测试3	16,937	1.85731		
32	测试1	19,580	1.6066	1.6416	4
	测试2	18,933	1.66151		
	测试3	18,988	1.65669		
64	测试1	18,851	1.66873	1.67719	3
	测试2	18,639	1.68771		
	测试3	18,779	1.67513		
128	测试1	19,992	1.57349	1.61262	5
	测试2	18,643	1.68735		
	测试3	19,947	1.57704		

窗口大小与性能关系分析:

1.窗口过小(1-4)

特征：接收窗口严重限制性能

分析：接收窗口1时，客户端每发送1个包就要等待ACK，RTT完全主导性能

2.快速上升(4-16)

特征：性能随窗口增加快速提升

分析：逐渐接近客户端发送能力上限

3.性能峰值(16)

特征：达到最佳性能点1.782 MB/s

分析：接收窗口=16时，服务端处理能力和客户端发送能力达到最佳平衡

4.性能稳定/下降(16-128)

特征：性能不再增长或轻微下降

分析：客户端发送窗口=32成为新瓶颈，更大接收窗口无益

4.3 传输文件实战

首先我们按要求配置路由器：丢包率3%-5%，延迟5ms-10ms。



随后在两个终端中分别运行命令

```
RDTServer 2333

RDTClient 127.0.0.1 8000 1.jpg
```

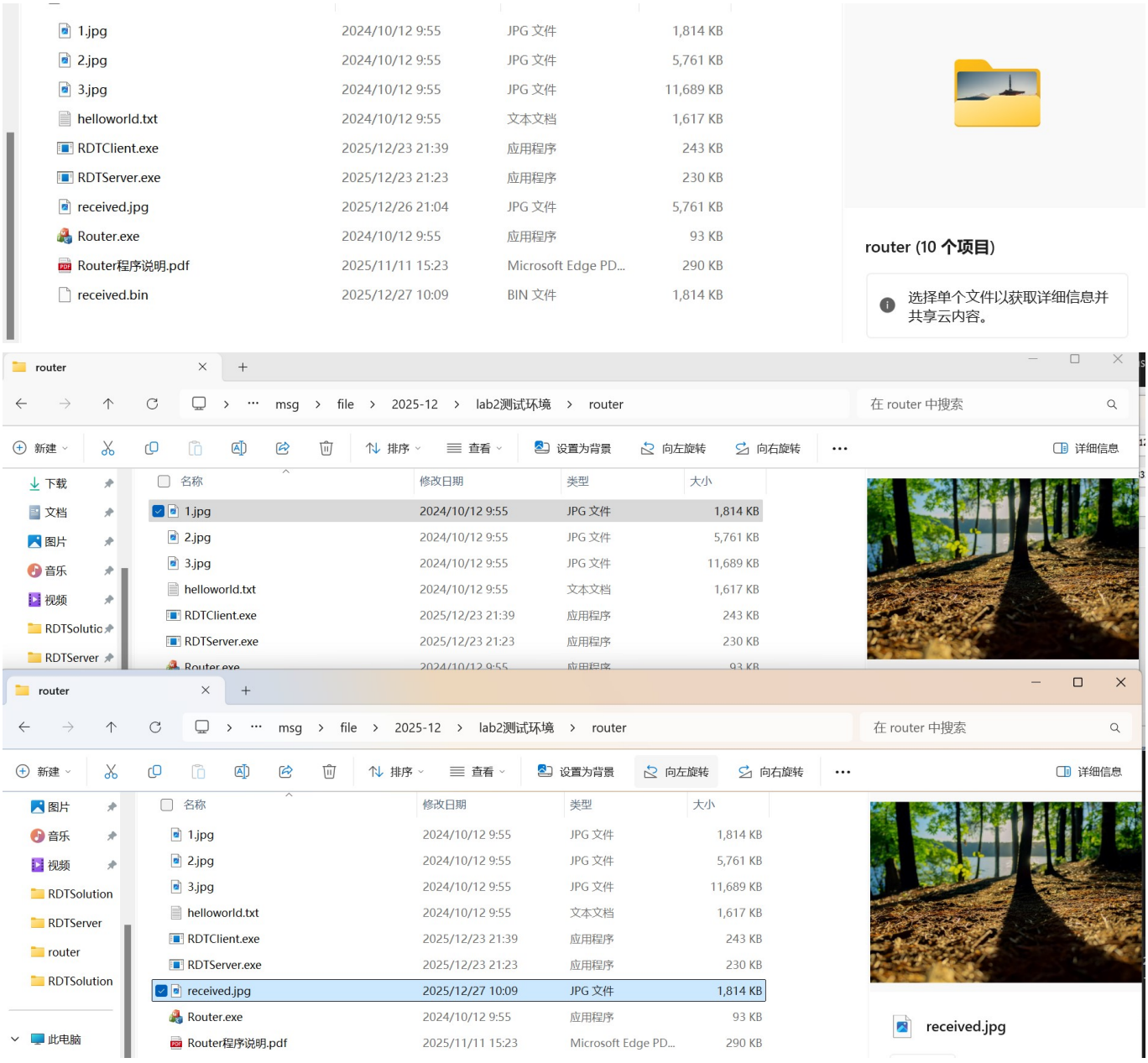
传输一下1.jpg这个图片，形成如下所示的网络：

```
+---------+ 127.0.0.1:8000 +------+ 127.0.0.1:2333 +---------+
|RDTClient|----------------|Router|----------------|RDTServer|
+---------+                +------+                +---------+
```

最后运行结果：

```
#服务端
========== Reception Statistics ==========
Total packets received: 2107
Truncated packets: 0
Max out-of-order buffered: 0
Output file size: 1857353 bytes
==========================================
```

```
#客户端
========== Transmission Statistics ==========
Total packets sent: 1859
File size: 1857353 bytes
Transmission time: 62691 ms
Throughput: 0.0296271 MB/s
Final cwnd: 6
Final ssthresh: 1
Total ACKs received: 2108
Client advertised window: 32
Server advertised window: 32
Final close state: 3
============================================
```



5. 具体实现详解

5.1 连接管理

5.1.1 三次握手建立连接

在客户端和服务端开始连接时，首先通过三次握手建立连接：

```
客户端(主动)                    服务端(被动)
    SYN(seq=0) ──────────────>
        <──────────────── SYN+ACK(seq=100, ack=1)
    ACK(seq=1, ack=101) ->
```

客户端实现握手的函数主要是client.cpp中的perform_handshake函数和server.cpp中的main函数，下面我将对关键的代码块进行解析：

```
    RDTHeader sh;
    memset(&sh, 0, sizeof(sh));
    sh.seq = 0;
    sh.ack = 0;
    sh.flags = RDT_SYN;
    sh.len = 0;
    sh.wnd = CLIENT_ADVERTISED_WINDOW;  // Client's advertised window
    sh.sack = 0;
```

这部分是客户端发送SYN请求时的设置，也就是第一次握手，首先设置了一个报文头并将其全部初始化为0，然后开始对其中的各个段进行初始化，最重要的是将flags设为RDT_SYN也就是0。随后这个报文头会被发出去，被服务端接收。

```
    // Three-way handshake
    while (!connected) {
      ......
        if (h.flags & RDT_SYN) {
            cout << "[handshake] received SYN, wnd=" << h.wnd << endl;

            // reply SYN+ACK
            RDTHeader rh;
            memset(&rh, 0, sizeof(rh));
            rh.seq = 100;
            rh.ack = h.seq + 1;
            rh.flags = RDT_SYN | RDT_ACK;
            rh.wnd = recv_wnd;
            fill_checksum(&rh, nullptr);
            RDTHeader neth = rh;
            hdr_hton(&neth);
            sendto(sockfd, (char*)&neth, sizeof(neth), 0, (sockaddr*)&cli,
clilen);

            cout << "[handshake] sent SYN+ACK, wnd=" << rh.wnd << endl;
        }
        else if ((h.flags & RDT_ACK) && h.ack) {
            connected = true;
```

```
            cout << "[handshake] complete, client wnd=" << h.wnd << endl;
        }
      ......
      }
```

这是服务端主函数中的一个检测三次握手的循环，服务端主要的处理三次握手的代码，其中h是服务端以字符串形式接收并复制后的报文头对象，其中如果h的flags的RDT_SYN位为1，则进入第二次握手的处理流程，将rh的flags段的RDT_SYN和RDT_ACK位置位1，并将ack段置1，发给客户端，这是第二次握手。

如果h的flags的RDT_ACK位为1且ack段不为零，代表这是第三次握手，成功建立连接。

```
    while (true) {
        int r = recvfrom(sockfd, buf, sizeof(buf), 0, (sockaddr*)&from, &fromlen);
        if (r > 0) {
            if (!verify_checksum_raw(buf, r)) continue;
            RDTHeader rh; memcpy(&rh, buf, sizeof(RDTHeader)); hdr_ntoh(&rh);
            if ((rh.flags & RDT_SYN) && (rh.flags & RDT_ACK)) {
                cout << "[handshake] received SYN+ACK, server wnd=" << rh.wnd <<
endl;
                // update receiver window from server
                receiver_wnd = rh.wnd;

                // send ACK
                RDTHeader ah;
                memset(&ah, 0, sizeof(ah));
                ah.seq = 1;
                ah.ack = rh.seq + 1;
                ah.flags = RDT_ACK;
                ah.wnd = CLIENT_ADVERTISED_WINDOW;  // Client's advertised window
                fill_checksum(&ah, nullptr);
                RDTHeader netah = ah; hdr_hton(&netah);
                sendto(sockfd, (char*)&netah, sizeof(netah), 0,
(sockaddr*)&servaddr, sizeof(servaddr));
                cout << "[handshake] sent ACK, wnd=" << ah.wnd << endl;
                return 0;
            }
        }
        ......
    }
```

这是client.cpp中perform_handshake函数的一部分，在发出第一次握手后，客户端就会开始等待服务端的回应，接收到后，先检查rh中的flags段的SYN和ACK标志位是否都是1，如果是，证明这是第二次握手，接下来将报文头的seq段设为1，并将flags的ACK位设为1，ack段设为101，发送给服务端，这是第三次握手。

三次握手完成后，应该能看到这样的输出：

```
D:\HuaweiMoveData\Users\26805\Desktop\RDTSolution\x64\Debug>RDTServer 2333
Server listening on port 2333, receiver window=32 segments
[handshake] received SYN, wnd=32
```

```
[handshake] sent SYN+ACK, wnd=32
[handshake] complete, client wnd=32

D:\HuaweiMoveData\Users\26805\Desktop\RDTSolution\x64\Debug>RDTClient 127.0.0.1
2333 input.bin
Starting connection to 127.0.0.1:2333
Parameters: MSS=1000 MAX_WINDOW=32 RTO=200ms LOSS=0 ClientWindow=32
[handshake] sent SYN, wnd=32
[handshake] received SYN+ACK, server wnd=32
[handshake] sent ACK, wnd=32
```

5.1.2 四次挥手断开连接

在客户端和服务端结束连接时，要通过四次挥手来结束连接：

```
客户端(主动关闭)            服务端
    FIN(seq=last+1) ———————>
        <————————————— ACK(ack=last+2)
        <————————————— FIN(当数据发完时)
    ACK ————————————————————>
```

```
// 在发送完所有数据包后发送FIN
Packet finp;
memset(&finp.hdr, 0, sizeof(finp.hdr));
finp.hdr.seq = (uint32_t)total;  // FIN的seq = 文件总字节数
finp.hdr.flags = RDT_FIN;        // 设置FIN标志
fill_checksum(&finp.hdr, nullptr);
packets.push_back(finp);  // 将FIN包加入发送队列
......
// 发送FIN时更新状态
if (p.hdr.flags & RDT_FIN && close_state == CLOSE_IDLE) {
    close_state = CLOSE_FIN_SENT;  // 进入"已发送FIN"状态
    cout << "[fin] sending FIN, entering close state" << endl;
}
```

这是client.cpp中发送FIN报文头的代码和发送后的状态更新，具体原理和上面的三次握手差不多，这里不再赘述。

```
// 收到客户端的FIN（四次挥手第一步）
if (h.flags & RDT_FIN) {
    if (!connection_closing) {
        cout << ">>> Received FIN, sending ACK and preparing to close" << endl;

        // 发送ACK确认客户端的FIN（第二步）
        send_ack(cli, h.seq + 1);

        // 标记开始关闭流程
```

```
            connection_closing = true;  // 进入半关闭状态

            // 注意：这里不立即break，继续处理可能有的乱序数据
        }
    }
    ......
    // 服务器在数据处理完后发送FIN（第三步）
    if (connection_closing && !fin_sent) {
        // 检查是否所有数据都已处理（简化检查：没有乱序包）
        if (out_of_order.empty()) {
            cout << "[fin] all data processed, sending our FIN" << endl;
            send_fin(cli);  // 发送服务器的FIN
        }
    }
```

这是server.cpp中用于处理客户端FIN和发送自己的ACK和FIN的代码。

```
// 收到服务器的FIN（第四步）
if (hdr.flags & RDT_FIN) {
    unique_lock<mutex> lk(mtx);
    cout << "[recv] received FIN from server, seq=" << hdr.seq << endl;
    received_server_fin = true;
    server_fin_seq = hdr.seq;

    // 发送ACK确认服务器的FIN（第四步的响应）
    RDTHeader ack_hdr;
    ack_hdr.ack = hdr.seq + 1;  // 确认服务器的FIN
    ack_hdr.flags = RDT_ACK;
    sendto(sockfd, (char*)&net_ack, sizeof(net_ack), 0,
          (sockaddr*)&servaddr, sizeof(servaddr));

    close_state = CLOSE_WAIT_FIN_ACK;  // 进入等待状态
}
```

这是client.cpp中用于处理服务端FIN和发送自己的ACK的代码。

```
// 服务器发送FIN后等待ACK
if (connection_closing && fin_sent) {
    cout << "[fin] waiting for ACK to our FIN..." << endl;
    this_thread::sleep_for(chrono::milliseconds(100));

    // 简化的超时处理
    static auto fin_sent_time = chrono::steady_clock::now();
    if (chrono::duration_cast<chrono::seconds>(
        chrono::steady_clock::now() - fin_sent_time).count() > 1) {
        cout << "[fin] timeout, assuming ACK lost, closing anyway" << endl;
        break;  // 超时后强制关闭
    }
}
```

这是server.cpp中用于处理客户端ACK并关闭所有线程的代码。

最后应该会有类似这样的输出:

```
D:\HuaweiMoveData\Users\26805\Desktop\RDTSolution\x64\Debug>RDTClient 127.0.0.1
2333 input.bin
......
[fin] sending FIN, entering close state
[send] FIN seq=31457280 wnd=32
[complete] All 31459 packets acknowledged
[fin] waiting for server FIN...
[recv] received FIN from server, seq=100
[fin] sent ACK for server FIN, ack=101

D:\HuaweiMoveData\Users\26805\Desktop\RDTSolution\x64\Debug>RDTServer 2333
......
[recv] FIN seq=31457280 ack=0 wnd=32 sack=0x0
>>> Received FIN, sending ACK and preparing to close
[fin] all data processed, sending our FIN
[fin] sent FIN, seq=100 ack=31457281
[fin] waiting for ACK to our FIN...
[recv] data seq=0 len=0 ack=101 wnd=32 sack=0x0
>>> Received ACK for our FIN, connection closed
Successfully wrote received.bin, size=31457280 bytes
```

5.1.3 连接异常处理

以下是server.cpp中处理连接空闲超时的代码:

```
// 连接空闲检测（300秒超时）
if (r <= 0) {
    this_thread::sleep_for(chrono::milliseconds(5));

    // 检查空闲时间
    if (chrono::duration_cast<chrono::seconds>(
        chrono::steady_clock::now() - last_active).count() > 300) {
        cout << "Timeout, closing connection" << endl;
        break;  // 关闭连接
    }
    continue;
}
```

以下是server.cpp中用于处理RST包的代码:

```
// 收到RST立即关闭连接
if (h.flags & RDT_RST) {
    cout << "[recv] RST received, closing connection immediately" << endl;
```

```
    break;  // 立即跳出主循环，关闭连接
}
```

以下是server.cpp中处理错误格式的包的代码：

```
// 包大小检查
if (r < (int)sizeof(RDTHeader)) {
    cout << "[error] packet too small: " << r << " bytes" << endl;
    truncated_packets++;
    continue;  // 丢弃无效包
}

// 包长度一致性检查
if (r < (int)(sizeof(RDTHeader) + h.len)) {
    cout << "[error] truncated packet" << endl;
    truncated_packets++;
    continue;  // 丢弃不完整包
}
```

5.2 校验和

这个RDT实现使用了标准的16位二进制反码求和作为校验和算法，覆盖整个RDTHeader（checksum字段置零）和所有payload数据。在校验前将数据转换为网络字节序以确保跨平台一致性。

核心校验和计算函数：

```
static inline uint16_t rdt_checksum(const void* data, size_t len) {
    const uint8_t* buf = (const uint8_t*)data;
    uint32_t sum = 0;
    size_t i = 0;

    // 步骤1：以16位（2字节）为单位累加
    while (i + 1 < len) {
        sum += (uint16_t)(buf[i] << 8 | buf[i + 1]);  // 将两个字节组合为16位
        while (sum >> 16) {
            sum = (sum & 0xFFFF) + (sum >> 16);
        }
        i += 2;
    }

    // 步骤2：处理奇数长度情况（最后一个字节）
    if (i < len) {
        sum += (uint16_t)(buf[i] << 8);  // 最后一个字节左移8位（补0）
        while (sum >> 16) {
            sum = (sum & 0xFFFF) + (sum >> 16);
        }
    }

    // 步骤3：取反得到最终校验和
```

```
    sum = ~sum;
    return (uint16_t)(sum & 0xFFFF);
}
```

函数传进去两个参数，data是指向要进行校验的数据段的起始地址，len是数据段长度，在步骤一，将数据段每两个字节分隔开并进行累加，如果有高16位进位的话，不直接丢弃这个进位，而是将其"回卷"加到计算结果的最低位上。在步骤二，算法处理了数据段是奇数个字节的情况，因为i始终是偶数，如果数据段是奇数，那么一定有一刻i + 1 = len且i < len，这时将这个字节扩展到16位，低位补零，累加到结果中，回滚进位。在步骤三，将sum取反并截取第十六位获得最终校验和。

在服务端接收了数据包后，会重新计算校验和，并与原先的校验和比较，以此完成验证，具体的操作在verify_checksum_raw函数里：

```
static inline bool verify_checksum_raw(const char* rawbuf, size_t rawlen) {
    // 1. 基本检查
    if (!rawbuf || rawlen < sizeof(RDTHeader)) return false;

    // 2. 从原始数据提取长度字段（小心：这是网络字节序！）
    uint16_t net_len;
    memcpy(&net_len, rawbuf + offsetof(RDTHeader, len), sizeof(uint16_t));
    size_t payload_len = ntohs(net_len);  // 转换为主机字节序

    // 3. 计算总长度
    size_t tot = sizeof(RDTHeader) + payload_len;
    if (rawlen < tot) return false;  // 包不完整

    // 4. 提取接收到的校验和
    uint16_t net_orig;
    memcpy(&net_orig, rawbuf + offsetof(RDTHeader, checksum), sizeof(uint16_t));
    uint16_t orig_checksum = ntohs(net_orig);  // 转换为主机字节序

    // 5. 创建临时缓冲区（将checksum字段置零）
    std::vector<char> tmp(tot);
    memcpy(tmp.data(), rawbuf, sizeof(RDTHeader));

    // 关键：将checksum字段置零，因为计算时不包含它
    tmp[offsetof(RDTHeader, checksum)] = 0;
    tmp[offsetof(RDTHeader, checksum) + 1] = 0;

    // 复制payload（如果有）
    if (payload_len) {
        memcpy(tmp.data() + sizeof(RDTHeader),
               rawbuf + sizeof(RDTHeader),
               payload_len);
    }

    // 6. 重新计算校验和
    uint16_t calc_checksum = rdt_checksum(tmp.data(), tot);

    // 7. 比较
```

```
    return calc_checksum == orig_checksum;
}
```

我们可以看到，这里面涉及到网络字节序向主机字节序的转换，也就是ntohs函数，其中n代表网络，h代表主机，s代表short，也就是16字节。这两种字节序主要涉及到大端地址和小端地址之间的转换，通常情况下，主机字节序是小端，也就是低位对应低地址，网络字节序是大端，也就是低位对应高地址。我们的RDT.h中也有类似的解析函数：

```
static inline void hdr_hton(RDTHeader* h) {
    h->seq = htonl(h->seq);
    h->ack = htonl(h->ack);
    h->flags = htons(h->flags);
    h->checksum = htons(h->checksum);
    h->wnd = htons(h->wnd);
    h->len = htons(h->len);
    h->sack = htonl(h->sack);
}

static inline void hdr_ntoh(RDTHeader* h) {
    h->seq = ntohl(h->seq);
    h->ack = ntohl(h->ack);
    h->flags = ntohs(h->flags);
    h->checksum = ntohs(h->checksum);
    h->wnd = ntohs(h->wnd);
    h->len = ntohs(h->len);
    h->sack = ntohl(h->sack);
}
```

回到verify_checksum_raw函数，这个函数是直接使用获得的网络原始数据进行验证，通过偏移找到两个重要的段：len和checksum。并将这两个段进行字节序转换后，读取全部信息并开始计算校验和，然而，它只对len和checksum进行了字节序转换，而其它部分没有，这是因为校验和的计算必须在统一的字节序中进行，我们在client.cpp中发送数据包时使用的是fill_checksum函数填充校验和：

```
static inline void fill_checksum(RDTHeader* h, const char* payload) {
    h->checksum = 0;
    // compute checksum over network-order header + payload
    RDTHeader tmp = *h;
    tmp.checksum = 0;
    // convert header to network order for checksum canonical form
    tmp.seq = htonl(tmp.seq);
    tmp.ack = htonl(tmp.ack);
    tmp.flags = htons(tmp.flags);
    tmp.checksum = htons(tmp.checksum);
    tmp.wnd = htons(tmp.wnd);
    tmp.len = htons(tmp.len);
    tmp.sack = htonl(tmp.sack);
    uint16_t cks = rdt_checksum(&tmp, sizeof(RDTHeader));
    if (h->len && payload) {
```

```
        size_t tot = sizeof(RDTHeader) + h->len;
        std::vector<char> buf(tot);
        memcpy(buf.data(), &tmp, sizeof(RDTHeader));
        memcpy(buf.data() + sizeof(RDTHeader), payload, h->len);
        cks = rdt_checksum(buf.data(), tot);
    }
    h->checksum = cks;
}
```

可以看到，他是在转换成网络字节序后计算的校验和，因此我们的接收方也要在网络字节序上计算校验和。

5.3 确认重传

该RDT协议采用了流水线传输与选择确认(SACK)相结合的机制，允许发送方连续发送多个数据包，接收方可以确认非连续到达的数据包。

5.3.1 流水线

流水线传输机制允许同时发送多个数据包而不必等待前一个包的确认，提高了信道利用率。

核心数据结构，数据包：

```
struct Packet {
    RDTHeader hdr;                    // 包头
    vector<char> data;                // 数据
    steady_clock::time_point sent_time; // 发送时间（用于RTO）
    bool acked = false;               // 确认状态
};
```

在文件发送的过程中，窗口是“滑动的”，管理代码如下：

```
// 客户端发送循环中的窗口控制
int allowed_window = min(cwnd, receiver_wnd);  // 实际可发送窗口
int in_flight = 0;  // 正在传输中的包数量

// 统计正在传输中的包（已发送但未确认）
for (size_t i = 0; i < packets.size(); ++i) {
    if (!packets[i].acked && packets[i].sent_time.time_since_epoch().count() > 0)
{
        in_flight++;
    }
}

// 流水线发送：只要窗口内有空位就继续发送
while (in_flight < allowed_window && next_pkt_idx < pkt_count) {
    // 发送第next_pkt_idx个包
    Packet& p = packets[next_pkt_idx];
    p.sent_time = steady_clock::now();
```

```
    if (p.hdr.len && !p.data.empty()) {
        // 发送数据包
        send_with_optional_loss(out.data(), (int)out.size(), &servaddr);
    }

    next_pkt_idx++;  // 移动发送指针
    in_flight++;     // 增加在途包计数
}
```

我们将实际窗口分成三部分：已确认数据包、在途数据包、待发数据包，实际可用的窗口受两个因素的影响：网络承载能力（cwnd），接收方窗口数量（receiver_wnd）。从而得出实际发送窗口allowed_window，其中，cwnd和receiver_wnd不是静态的，cwnd会服从RENO调度算法，在到达阈值前，处于慢启动阶段，在到达阈值后，处于拥塞避免阶段，发生丢包后，进入快速恢复阶段。receiver_wnd会在服务端ACK确认收到信息中发送过来。

我们在每次发送时，都会发送allowed_window - in_flight个包，也就是说，每次发送前客户端线程都要确认以上的内容，而服务端已经确认了哪些包，会在包头的sack段发来。

5.3.2 选择确认

接收方通过SACK位图（RDTHeader::sack）告知发送方哪些数据包已经成功接收，即使它们不连续。这减少了不必要的重传。

每当客户端若干个数据包发送过来，服务端都要回一个确认包，告知客户端累计确认、选择确认和窗口数量情况，其中累计确认就是包头的ack段，代表服务端接下来期望接收到的数据偏移量，而选择确认就是本次发送的若干个数据包有哪些进入了服务端窗口，避免重复发送。

```
    RDTHeader h;
    memset(&h, 0, sizeof(h));
    h.seq = 0;
    h.ack = acknum;
    h.flags = RDT_ACK;
    h.wnd = recv_wnd;

    // build sack bitmap for next 32 segments beyond acknum
    uint32_t mask = 0;
    for (int i = 0; i < RDT_MAX_SACK_BITS; i++) {
        uint32_t s = acknum + i * RDT_MSS;
        if (out_of_order.count(s))
            mask |= (1u << i);
    }
    h.sack = mask;
```

这是服务端生成sack段的函数，普通的ACK只能告诉发送方“这个序列号之前的数据我都收到了”。但如果数据包乱序到达（比如收到了第2包，没收到第1包），发送方通常会重传第1包以及之后的所有包。在这个函数中，容器out_of_order存放的是接收方已经收到的、但是序列号大于acknum的“乱序数据包”，如果接收方确实收到了序列号为s的包，就在位图mask的第i位上置1，其中s = acknum + i * RDT_MSS。

```
sack_bitmap = hdr.sack;  // 获取SACK位图

if (sack_bitmap > 0) {
    for (int i = 0; i < RDT_MAX_SACK_BITS; ++i) {
        if (sack_bitmap & (1u << i)) {  // 检查第i位
            // 计算被SACK确认的包序列号
            uint32_t seq = acknum + (uint32_t)i * (uint32_t)SENDER_MSS;

            // 标记对应包为已确认
            for (auto& p : packets) {
                if (!p.acked && p.hdr.seq == seq) {
                    p.acked = true;  // 选择确认
                    cout << "[sack] packet seq=" << p.hdr.seq << " selectively
acked" << endl;
                    break;
                }
            }
        }
    }
}
```

这是client.cpp中解析服务端sack段的代码，标记为已确认后就会进入到已确认数据包这部分。

5.3.3 重传策略

```
// 关键变量:
uint32_t lastAck = 0;      // 上一个ACK号
int dupAckCount = 0;       // 重复ACK计数器

else if (acknum == lastAck) {
    dupAckCount++;
    if (dupAckCount == 3) {
        // fast retransmit
        ssthresh = max(1, cwnd / 2);
        cwnd = ssthresh + 3;
        cout << "[congestion] fast retransmit: ssthresh=" << ssthresh << " cwnd="
<< cwnd << endl;

        // retransmit earliest unacked packet
        for (auto& p : packets) {
            if (!p.acked) {
                p.sent_time = steady_clock::now();
                if (p.hdr.len && !p.data.empty()) {
                    vector<char> out(sizeof(RDTHeader) + p.data.size());
                    RDTHeader th = p.hdr;
                    th.wnd = CLIENT_ADVERTISED_WINDOW; // Use updated client
window
                    fill_checksum(&th, p.data.data());
                    hdr_hton(&th);
                    memcpy(out.data(), &th, sizeof(RDTHeader));
```

```
                    memcpy(out.data() + sizeof(RDTHeader), p.data.data(),
p.data.size());
                    sendto(sockfd, out.data(), (int)out.size(), 0,
(sockaddr*)&servaddr, sizeof(servaddr));
                    cout << "[retransmit] fast retransmit seq=" << p.hdr.seq <<
endl;
                }
                break;
            }
        }
    }
}
```

当同一个ACK累计达到三次时，触发快速重传策略。

```
// 1. 超时时间设置
int RTO_MS = 200;  // 默认200毫秒，可通过--rto参数调整
// 2. 每个数据包记录发送时间
struct Packet {
    // ...
    steady_clock::time_point sent_time;  // 记录发送时间
    bool acked = false;                   // 是否已确认
};
// 3. 发送数据包时记录时间
p.sent_time = steady_clock::now();  // 发送时记录当前时间

// 检查最早未确认数据包的超时
uint32_t earliest_seq = UINT32_MAX;
int earliest_idx = -1;

// 找到最早未确认的数据包
for (size_t i = 0; i < packets.size(); ++i) {
    if (!packets[i].acked) {
        if (packets[i].hdr.seq < earliest_seq) {
            earliest_seq = packets[i].hdr.seq;
            earliest_idx = (int)i;
        }
    }
}

// 如果发现超时
if (earliest_idx >= 0) {
    auto& p = packets[earliest_idx];
    // 计算从发送到现在的时间差
    auto elapsed = duration_cast<milliseconds>(
        steady_clock::now() - p.sent_time).count();

    if (elapsed > RTO_MS) {  // 超过RTO时间
        // 执行超时重传
        // ...
    }
}
```

当发送一个数据包后，如果在预定时间内没有收到确认，就重传该数据包，此为超时重传。

5.4 流量控制

流量控制确保发送方不会淹没接收方，通过接收窗口(receiver_wnd)来限制发送方的发送速率。

```
//发送方(client.cpp)
// 关键变量:
int receiver_wnd = MAX_WINDOW;  // 从接收方（服务器）的ACK中获取
int cwnd = 1;                   // 拥塞窗口
int allowed_window = min(cwnd, receiver_wnd);  // 实际允许的窗口大小

//接收方(server.cpp)
// 关键变量:
int recv_wnd = 32;  // 接收窗口大小（默认为32个数据段）

// 在发送ACK时包含窗口信息:
void send_ack(const SOCKADDR_IN& cli, uint32_t acknum) {
    RDTHeader h;
    h.wnd = recv_wnd;  // 将接收窗口大小告知发送方
    // ... 发送ACK
}
```

具体的工作流程:

1. 初始化:

- 双方建立连接时互相确认窗口大小

2. 发送阶段:

- 发送方根据接收方通告的receiver_wnd限制发送速率
- 每个ACK都会更新接收窗口信息

3. 窗口更新:

- 接收方处理数据后发送ACK，包含当前可用窗口
- 发送方根据ACK更新receiver_wnd

4. 拥塞控制与流量控制的协调:

- 实际窗口 = min(拥塞窗口, 接收窗口)，既防止网络拥塞，又防止接收方缓冲区溢出

5.5 RENO算法

```
if (acknum > lastAck) {
    dupAckCount = 0;
```

```
        lastAck = acknum;
        send_base = acknum;

        // congestion control with limit
        if (cwnd < ssthresh) { // slow-start
            cwnd += 1;
            // Limit cwnd growth to avoid exceeding receiver capability
            if (cwnd > receiver_wnd * 2) {
                cwnd = receiver_wnd * 2;
            }
            if (total_acks_received % 50 == 0) {
                cout << "[congestion] slow-start: cwnd=" << cwnd << endl;
            }
        }
        else { // congestion avoidance
            static int ack_acc = 0;
            ack_acc++;
            if (ack_acc >= cwnd) {
                cwnd++;
                // Limit cwnd growth
                if (cwnd > receiver_wnd * 2) {
                    cwnd = receiver_wnd * 2;
                }
                ack_acc = 0;
                if (total_acks_received % 100 == 0) {
                    cout << "[congestion] avoidance: cwnd=" << cwnd << endl;
                }
            }
        }
    }
```